Lambdas and Power Functions: pandas Notes for Daily Work

Nov 2, 2026 · @tanvik

Overview

A lambda is a one-line, unnamed function. It's best for small custom logic in places that take a function: `key=`, `.assign()`, `.loc[]`, `rename()`. Inside `.apply()` it runs once per row in plain Python, so it's usually the slow choice. On 500,000 rows, a row-wise `apply` took 4.7 seconds where plain column arithmetic took 5 milliseconds.

The speed ladder, fastest first:

1. Whole-column arithmetic and comparisons: `s * 2`, `df.a + df.b`, `s > 60`.
2. NumPy and pandas building blocks: `np.where`, `np.select`, `pd.cut`, `pd.to_datetime`, `pd.to_timedelta`.
3. Grouped work with built-in names: `groupby().agg('sum')`, `transform('max')`, `shift()`.
4. Lookups and text: `.map(dict)`, `.str.replace(regex)`, `.str.extract()`.
5. `.apply(lambda ...)` on one column.
6. `df.apply(..., axis=1)` and `for` loops over rows: last resort.

How to use these notes: the decision diagram tells you which tool to reach for. The middle sections teach lambdas and the power functions, each with code on data like yours. The before-and-after section rewrites patterns from your own scripts, and the practice tasks at the end lock it in.

All timings were measured in pandas 3.0.2 on 500,000 rows. Exact numbers vary by machine, but the order holds.

Which tool to reach for



which tool to reach for · 5 checks before apply

Work down the left column and stop at the first "yes". Only a calculation that fails all five checks belongs in `.apply(lambda ...)`.

Lambda basics in Python

`lambda inputs: expression` makes a function in one line. It has no name, no `return`, and only one expression. Anything longer belongs in a `def`.

```
double = lambda x: x * 2           # same as: def double(x): return x * 2
label = lambda secs: "long" if secs > 60 else "short"   # if/else must be the inline form
double(21), label(75)              # (42, 'long')
```

Where lambdas earn their place: the `key=` argument

`sorted`, `max`, `min` and `.sort()` take a `key` function that says **what to compare**. This is the most useful lambda in daily work.

```
from pathlib import Path
files = list(Path(r"C:\...\Team List").iterdir())

newest  = max(files, key=lambda f: f.stat().st_mtime)      # newest file
biggest = max(files, key=lambda f: f.stat().st_size)       # largest file
by_name = sorted(files, key=lambda f: f.name.lower())      # A to Z, ignoring
capitals

agents = [("riya", 41), ("amrut", 58), ("neha", 33)]
sorted(agents, key=lambda a: a[1], reverse=True)           # by calls, most
first
```

`map` and `filter` (usually a comprehension reads better)

```
ids = [" C105 ", "C999", "No New Id"]
list(map(lambda x: x.strip(), ids))                # ['C105', 'C999', 'No
New Id']
list(filter(lambda x: x.startswith("C"), ids))     # ['C999']: ' C105 '
starts with a space

[x.strip() for x in ids]                           # same as the map,
clearer
[x for x in ids if x.strip().startswith("C")]      # same idea as the
filter
```

Lambda or def?

Use a lambda	Use a def
One short expression	Several steps, or an `if` / `elif` chain
Used once, right where it's written	Reused in several places
`key=`, `.assign()`, `.loc[]`, `rename(columns=...)`	Needs a docstring, a test, or a clear name in error messages

A trap: lambdas made in a loop

```
funcs = [lambda x: x + i for i in range(3)]
[f(10) for f in funcs]              # [12, 12, 12]: every lambda sees the
LAST i

funcs = [lambda x, i=i: x + i for i in range(3)]
[f(10) for f in funcs]              # [10, 11, 12]: i=i freezes each value
```

A lambda looks up outside variables when it runs, not when it's made.

Lambdas in pandas

The good lambdas in pandas receive a **whole column or a whole DataFrame**, so the work inside stays vectorised. The slow ones receive **one value or one row at a time**.

Good: the lambda gets the whole DataFrame or column

```
out = (calls
       .rename(columns=lambda c: c.strip())                          #
'Dialer Name ' -> 'Dialer Name'
       .assign(talk_s=lambda d: d['Talk Time'].dt.total_seconds(),
               connected=lambda d: d['Call Status'].eq('connected'),
               long_call=lambda d: d['connected'] & (d['talk_s'] > 30))
       .loc[lambda d: d['talk_s'] >= 0]                              # filter
mid-chain
       .sort_values('Dialer Name', key=lambda s: s.str.lower()))     # sort
ignoring capitals
```

Where	What `d` or `s` is	Why it helps
`.assign(new=lambda d: ...)`	The DataFrame at that point in the chain	Uses columns made earlier in the same chain; `long_call` uses `connected` above
`.loc[lambda d: ...]`	The current DataFrame	Filters without saving a temporary variable
`.rename(columns=lambda c: ...)`	One column **name** (text)	Cleans every header at once

Where	What `d` or `s` is	Why it helps
`sort_values(key=lambda s: ...)`	The whole sort column	Sorts on a cleaned version, not the raw text

Good: the lambda gets one group

```
calls.groupby('CRM ID').agg(
    first_call=('Call Start Time', 'min'),
    numbers=('Number', lambda s: ', '.join(s.unique())))      # pandas'
STRING_AGG

calls.groupby('lead_id').filter(lambda g: len(g) >= 5)        # keep groups
with 5+ rows
```

With 50 agents, the lambda runs 50 times, not once per call. Joining text per group has no built-in name, so a lambda is the right tool there.

Slow: the lambda gets one value or one row

```
calls['label'] = calls['talk_s'].apply(lambda x: 'long' if x > 60 else
'short')   # once per row
calls['total'] = calls.apply(lambda r: r['talk_s'] + r['hold_s'], axis=1)
# once per row, slowest
```

Both have fast versions (`np.where`, `calls['talk_s'] + calls['hold_s']`); the measured section shows by how much. Keep `.apply` for logic with no vectorised form, such as calling your own Python function on short text.

`.pipe()`: your own function in the middle of a chain

```
def add_slot(d, start_hour=7):
    return d.assign(slot=(d['Call Start Time'].dt.hour -
start_hour).clip(lower=0))

calls.pipe(add_slot, start_hour=8)        # same as add_slot(calls,
start_hour=8)
```

`.pipe` keeps a long chain readable top to bottom, and the step can be reused in every report.

Share of the group: transform with a name, not a lambda

```
calls['share'] = calls['talk_s'] / calls.groupby('CRM ID')
['talk_s'].transform('sum')
```

`transform(lambda s: s / s.sum())` gives the same result, but `'sum'` is the built-in fast path.

Measured: lambda versus vectorised

The biggest wins are row-wise `apply` (900 times slower) and simple arithmetic through a lambda (180 times slower). Text is the exception: there, a plain lambda beat `.str.split()`.

Task	Slow version	Time	Fast version	Time	Faster by
Add two columns	`df.apply(lambda r: r.a + r.b, axis=1)`	4,718 ms	`df.a + df.b`	5 ms	934×
Multiply a column	`s.apply(lambda x: x * 2)`	105 ms	`s * 2`	0.6 ms	182×
Three bands	`s.apply(band)` with if/elif	73 ms	`np.select([...], [...], 'long')`	5.5 ms	13×
Two labels	`apply(lambda x: 'Yes' if x > 60 else 'No')`	78 ms	`np.where(s > 60, 'Yes', 'No')`	7.2 ms	11×
Three bands	`s.apply(band)`	78 ms	`pd.cut(s, bins, labels=...)`	10 ms	8×
Conditional sum per agent	`agg(lambda x: x[df.loc[x.index, ...] == ...].sum())`	96 ms	`.where(cond, 0)` then `groupby().sum()`	15 ms	6×
Count per agent	`agg(lambda x: len(x))`	18 ms	`groupby().size()`	9.6 ms	2×
Dict lookup	`apply(lambda x: lookup.get(x))`	75 ms	`s.map(lookup)`	39 ms	2×
Parse `HH:MM:SS`	`apply(to_td)` with split	704 ms	`pd.to_timedelta(s)`	439 ms	1.6×

Text: measure before assuming

Getting the second number out of `'917557160434 | 919810007919'`, 500,000 times:

Method	Time
`s.str.replace(r'^.*|\s*', '', regex=True)`	57 ms
`s.apply(lambda x: x.split('|')[1].strip())`	137 ms
`s.str.extract(r'|\s*(\d+)', expand=False)`	227 ms
`s.str.partition('|')[2].str.strip()`	427 ms
`s.str.split('|').str[1].str.strip()`	824 ms

Regex methods run inside pandas' compiled string engine, so they're fast. `.str.split()` builds a Python list for every row first, which makes it the slowest. All five give identical results.

How to time your own code

```
%timeit df['talk_s'].apply(lambda x: x * 2)      # in a notebook cell
%timeit df['talk_s'] * 2
```

Under 100,000 rows the difference rarely matters. Over a million it decides whether a report takes seconds or minutes.

Replace these lambdas: patterns from your own scripts

Each pattern below appeared in the dialer notebook or the ledger scripts. The "after" version gives the same result, faster and in fewer lines.

1. Two labels

```
# before
df['Call Status'] = df['Call Status'].str.lower().apply(lambda x: 'connected'
if x == 'answered' else 'not connected')
# after
df['Call Status'] = np.where(df['Call Status'].str.lower().eq('answered'),
'connected', 'not connected')
```

2. Bands from an if/elif function

```
# before
def attendance(s):
    if s < 4.5 * 3600: return 'Absent'
    elif s < 6 * 3600: return 'Half Day'
    elif s < 8.5 * 3600: return 'Warning'
    return 'Present'
summary['Attendance'] = summary['login_s'].apply(attendance)

# after: the first true condition wins, like if/elif
s = summary['login_s']
summary['Attendance'] = np.select([s < 4.5 * 3600, s < 6 * 3600, s < 8.5 *
3600],
                                  ['Absent', 'Half Day', 'Warning'],
default='Present')
```

3. Parsing durations by hand

```
# before
def normalize_talk_time(t):
    h, m, s = str(t).split(':')
    return pd.Timedelta(hours=int(h), minutes=int(m), seconds=int(s))
df['Talk Time'] = df['Talk Time'].apply(normalize_talk_time)
# after: also turns blanks into NaT instead of crashing
df['Talk Time'] = pd.to_timedelta(df['Talk Time'].astype(str).str.strip(),
errors='coerce')
```

4. Conditional counts inside groupby

```
# before: looks rows up again for every agent
summary = df.groupby('CRM ID').agg(
    connected=('Call Status', lambda x: (df.loc[x.index, 'Call Status'] ==
'connected').sum()))
# after: build the flag once, then a plain sum
summary = (df.assign(connected=df['Call Status'].eq('connected'))
             .groupby('CRM ID').agg(connected=('connected', 'sum')))
```

5. Cleaning IDs

```
# before
led['ClientID'] = led['ClientID'].apply(lambda x: str(x).strip())
# after
led['ClientID'] = led['ClientID'].astype(str).str.strip()
```

6. Row-wise arithmetic

```
# before
df['Total'] = df.apply(lambda r: r['Queue'] + r['Answer'], axis=1)
# after
df['Total'] = df['Queue'] + df['Answer']
```

7. Date filters

```
# before
led_y = led[led['CreatedAt'].apply(lambda d: d.date() == yesterday.date())]
# after: a half-open range, compared on the whole column at once
led_y = led[(led['CreatedAt'] >= yesterday) & (led['CreatedAt'] < today)]
```

8. Hour slots

```
# before
df['Interval'] = df['Call Start Time'].apply(lambda t: slot_label(t.hour))
# after: slot positions as numbers, then pick labels from an array
idx = (df['Call Start Time'].dt.hour - 7).clip(0, len(SLOT_LABELS) - 1)
df['Interval'] = np.array(SLOT_LABELS)[idx]
```

Method chaining: raw file to summary in one read

Wrap the steps in brackets and start each line with a dot. The code then reads top to bottom like a recipe, with no `df2`, `df3` or `temp` variables to mix up. This chain was tested on a practice Stringee export: 353 rows read, 335 after cleaning, then one row per agent.

```
def peek(d, label=''):
    print(f"{label}: {len(d):,} rows")
    return d                                    # must return d so the chain
continues
```

```
summary = (
    pd.read_excel(path, dtype={'Account': str, 'Customer number': str})
      .pipe(peek, 'read')
      .rename(columns=lambda c: c.strip())
      .assign(start=lambda d: pd.to_datetime(d['Start time']).dt.floor('s'),
              talk=lambda d: pd.to_timedelta(d['Answer
duration'].astype(str), errors='coerce'),
              agent=lambda d: d['Account'].str.replace(r'@.*', '',
regex=True).str.strip().str.lower(),
              connected=lambda d: d['Call status'].eq('Answered'))
      .loc[lambda d: d['agent'].notna() & d['agent'].ne('---')]
      .drop_duplicates(subset=['agent', 'Customer number', 'start'])
      .pipe(peek, 'cleaned')
      .assign(date=lambda d: d['start'].dt.normalize(),
              talk_s=lambda d:
d['talk'].dt.total_seconds().where(d['connected'], 0))
      .groupby(['date', 'agent'], as_index=False)
      .agg(dialed=('connected', 'size'), connected=('connected', 'sum'),
talk_s=('talk_s', 'sum'))
      .sort_values(['date', 'connected'], ascending=[True, False])
)
```

Why it's faster to work this way

- **No stale variables.** The `df` overwritten on 30 Oct can't happen; each run starts from the file.
- **Every step is one line**, so adding, removing or reordering a step is one edit.
- **`lambda d:` in `.assign` and `.loc`** always sees the data as it is at that line, including columns made just above.

Debugging a chain

- **Check row counts:** add `.pipe(peek, 'label')` between steps.
- **Find the broken step:** comment out lines from the bottom up and run until the error disappears.
- **Look at the data mid-chain:** end the chain early with `.head()` while building it.
- **Never use `inplace=True` in a chain.** It returns `None`, so the next `.` fails.

Power functions: conditions and buckets

These replace almost every `apply(lambda x: ... if ... else ...)`. Pick by the shape of the rule.

Rule	Function	Example
Two outcomes	`np.where(cond, a, b)`	Connected or not
Several outcomes, first match wins	`np.select([c1, c2], [a, b], default=c)`	Attendance, client tier
Number ranges into labels	`pd.cut(s, bins, labels)`	Talk-time bands
Equal-sized groups	`pd.qcut(s, q, labels)`	Balance halves or quartiles
Floor or cap a value	`s.clip(lower=0)`	Negative gaps to 0
Blank where a rule holds	`s.mask(cond)` / keep where it holds: `s.where(cond)`	Hide negative balances
First non-blank of two columns	`a.fillna(b)` or `a.combine_first(b)`	ARC ID, else Zynixx ID
Inside a range	`s.between(lo, hi)`	30 s to 5 min
One of a list	`s.isin([...])`	Unreachable dispositions

Talk-time bands with `pd.cut`

```
df['talk_band'] = pd.cut(df['talk_s'], bins=[-1, 0, 30, 300, np.inf],
                         labels=['No talk', 'Under 30s', '30s to 5m', 'Over
5m'])
df['talk_band'].value_counts(sort=False)       # bands in their natural order
```

Each bin includes its right edge: `(0, 30]` holds 30 but not 0. Start at `-1` so 0 lands in the first band.

Client tiers with `np.select`

```
b = df['balance']
df['tier'] = np.select([b >= 10_000, b >= 500], ['VIP', 'Active'],
default='Small')
```

Watch the default: a blank balance fails every condition and lands in `default`. In the test, a NaN balance became `'Small'`. Add `b.isna()` as the first condition, with label `'Unknown'`, when blanks are possible.

where, mask, clip and fillna

```
df['balance_fixed'] = df['balance'].clip(lower=0)             # -50 becomes
0
df['masked']        = df['balance'].mask(df['balance'] < 0)   # -50 becomes
blank
df['client_id']     = df['arc_id'].fillna(df['zynixx_id'])    # ARC ID, else
Zynixx ID
df['in_range']      = df['talk_s'].between(30, 300)           # both ends
included
```

`where` keeps values where the condition is True; `mask` is the opposite and blanks them. Both work on the whole column, so they replace a lambda with an `if` inside.

Power functions: groups and windows

Most "per client" or "per agent" questions are one of these. Sort first whenever order matters (first, previous, running total).

Question	Function	Result shape
One number per group	`groupby().agg(name=('col', 'sum'))`	One row per group
Group's value on every row	`groupby()['col'].transform('max')`	Same rows as before
Position inside the group	`groupby().cumcount() + 1`	1, 2, 3 per group
Previous row's value	`groupby()['col'].shift()`	Same rows

Question	Function	Result shape
Change since previous row	`groupby()['col'].diff()`	Same rows
Running total	`groupby()['col'].cumsum()`	Same rows
Rank inside the group	`groupby()['col'].rank(method='first')`	Same rows
Row holding the max	`df.loc[groupby()['col'].idxmax()]`	One row per group
Top N per group	`sort_values(...).groupby(...).head(N)`	N rows per group
Keep only some groups	`groupby().filter(lambda g: len(g) >= 5)`	Rows of those groups
Per day or week	`groupby(pd.Grouper(key='date', freq='D'))`, `resample('W')`	One row per period
Moving average	`s.rolling(7, min_periods=1).mean()`	Same rows

Per client, in date order

```
led = led.sort_values(['ClientID', 'CreatedAt'])
g = led.groupby('ClientID')

led['txn_no']          = g.cumcount() + 1                    # 1st, 2nd, 3rd
ledger row
led['prev_balance']    = g['Balance'].shift()
led['balance_change']  = g['Balance'].diff()                 # this row minus
the previous
led['days_since_prev'] = g['CreatedAt'].diff().dt.days
led['latest']          = g['CreatedAt'].transform('max')
```

Saving `g = led.groupby(...)` once and reusing it keeps the lines short. The grouping is worked out once.

Per day, with a 7-day moving average

```
daily = (led[led['LedgerType'].eq('BILL')]
           .groupby(pd.Grouper(key='CreatedAt',
freq='D')).size().rename('bills'))
daily_avg = daily.rolling(7, min_periods=1).mean().round(1)
```

`pd.Grouper(freq='D')` groups a date-time column by calendar day without making a separate date column. `freq='W'` gives weeks ending Sunday, and `'MS'` gives months.

Top 2 longest calls per agent

```
top2 = (calls.assign(talk_s=calls['Talk Time'].dt.total_seconds())
             .sort_values('talk_s', ascending=False)
             .groupby('CRM ID').head(2))
```

Running deposits per client

```
deposits = (led[led['LedgerType'].eq('DEPOSIT')]
              .assign(cum_deposit=lambda d: d.groupby('ClientID')
['Credit'].cumsum()))
```

All of these ran on the practice ledger and dialer data.

Power functions: reshaping and joining

`crosstab` alone would have built most of the Manager pivot in one line, totals included.

Need	Function
Counts of two categories against each other, with totals	`pd.crosstab(a, b, margins=True)`
Shares instead of counts	`pd.crosstab(a, b, normalize='index')`
Any aggregation, with a Total row and column	`pivot_table(..., margins=True, margins_name='Total')`
Wide columns into rows	`df.melt(id_vars=..., var_name=..., value_name=...)`
A list in a cell into one row each	`df.explode('col')`

Need	Function
Find rows that didn't match	`merge(..., how='outer', indicator=True)`
Stop a merge that would repeat rows	`merge(..., validate='many_to_one')`
Stack tables and label each	`pd.concat({'Stringee': a, 'Uno': b}, names=['Source'])`

Traded or not, per manager, in one line each

```
pd.crosstab(cf['Manager'], cf['Oct Trading'], margins=True,
margins_name='Total')     # counts + totals
pd.crosstab(cf['Manager'], cf['Oct Trading'], normalize='index').round(2)
# share per manager
```

`crosstab` counts rows. If a client can appear twice, use `pivot_table(index='Manager', columns='Oct Trading', values='Client ID', aggfunc='nunique', margins=True)` instead.

Wide hourly report to long, for charts and further grouping

```
long = hourly.melt(id_vars='CRM ID', var_name='Slot', value_name='Calls')
# CRM ID | Slot        | Calls
# a      | 9 to 10 AM  | 5
# a      | 10 to 11 AM | 7
```

Both numbers in Caller Ids, one row each

```
ex = (ids.assign(number=ids['Caller Ids'].str.split('|'))
         .explode('number')
         .assign(number=lambda d: d['number'].str.strip()))
```

The original row's index repeats after `explode`; add `.reset_index(drop=True)` when it matters.

Who didn't match the team list

```
m = calls.merge(team, on='Dialer Name', how='outer', indicator=True)
m['_merge'].value_counts()          # both / left_only (no team row) /
right_only (no calls)
m[m['_merge'] == 'left_only']       # the Not_Found_Users list
```

`validate='many_to_one'` raises `MergeError: Merge keys are not unique in right dataset` instead of silently repeating calls.

Python built-ins that save time

These work on lists, dicts and file names: the glue around your pandas code.

Tool	What it does	Example	Result
List comprehension	Build a list in one line	`[f.name for f in files if f.suffix == '.csv']`	CSV names
Dict comprehension	Build a dict in one line	`{c: c.lower().replace(' ', '_') for c in cols}`	A rename map
`zip`	Pair two lists	`dict(zip(names, paths))`	`{'Summary': 's.csv', ...}`
`enumerate`	Count while looping	`for i, f in enumerate(files, start=1):`	1, 2, 3 …
`any` / `all`	Is one / are all true?	`any(x > 31 for x in days)`	`True` or `False`
`Counter`	Count items	`Counter(disps).most_common(2)`	`[('No Answer', 3), ('Zero', 1)]`
`defaultdict(list)`	Group values without checking keys	`by_agent[agent].append(num)`	`{'riya': ['91', '93'], ...}`

Tool	What it does	Example	Result
`functools.reduce`	Apply a step across a list	Merge many tables on one key	One merged table

f-string formats you'll use every day

```
n, share, t = 1234567.891, 0.4567, pd.Timestamp('2026-10-28 14:05:09')
f"{n:,.2f}"               # '1,234,567.89'   thousands separator, 2 decimals
f"{share:.1%}"            # '45.7%'          a fraction as a percentage
f"{t:%d %b %Y %H:%M}"     # '28 Oct 2026 14:05'
f"{42:05d}"               # '00042'          pad with zeros
f"{len(df):,} rows"       # '20,808 rows'
```

Merging several tables on one key

```
from functools import reduce
tables = [summary, talk, gaps, login]                # all have 'CRM ID'
report = reduce(lambda left, right: left.merge(right, on='CRM ID',
how='left'), tables)
```

`reduce` merges the first two tables, then merges that result with the third, and so on.

Write your own helper once, use it everywhere

```
def clean_ids(s):
    """Text IDs: no spaces, no '.0' from floats, blanks kept blank."""
    return s.astype('string').str.strip().str.replace(r'\.0$', '',
regex=True)

led['ClientID'] = clean_ids(led['ClientID'])
cust['Client ID'] = clean_ids(cust['Client ID'])
```

Save helpers like this in one file (for example `nexora_helpers.py`) and `from nexora_helpers import clean_ids` in every notebook, the same way `get_final_ledger` works.

The same ideas in SQL

You write both every day, and nearly every pandas tool above has a SQL twin. Learning them in pairs halves the effort.

Idea	pandas	PostgreSQL
Two outcomes	`np.where(c, a, b)`	`CASE WHEN c THEN a ELSE b END`
First match wins	`np.select([c1, c2], [a, b], default=d)`	`CASE WHEN c1 THEN a WHEN c2 THEN b ELSE d END`
First non-blank	`a.fillna(b)`	`COALESCE(a, b)`
Blank where a rule holds	`s.mask(s == 0)`	`NULLIF(s, 0)`
Count only some rows	`cond.sum()` / `.where(cond, 0)` then `sum`	`COUNT(*) FILTER (WHERE cond)`
Distinct count	`nunique()`	`COUNT(DISTINCT x)`
Join text per group	`agg(lambda s: ', '.join(s))`	`STRING_AGG(x, ', ' ORDER BY y)`
Group value on every row	`transform('max')`	`MAX(x) OVER (PARTITION BY g)`
Position in group	`cumcount() + 1`	`ROW_NUMBER() OVER (PARTITION BY g ORDER BY t)`
Previous row	`shift()`	`LAG(x) OVER (PARTITION BY g ORDER BY t)`
Running total	`cumsum()`	`SUM(x) OVER (PARTITION BY g ORDER BY t)`
Latest row per key	`sort_values(...).drop_duplicates(key)`	`DISTINCT ON (key) ... ORDER BY key, t DESC`
Filter groups	`groupby().filter(lambda g: len(g) >= 5)`	`HAVING COUNT(*) >= 5`

Idea	pandas	PostgreSQL
Number bands	`pd.cut(s, bins, labels)`	`CASE` ranges, or `WIDTH_BUCKET(s, lo, hi, n)`
Text pattern	`str.extract(r'(\d+)')`	`SUBSTRING(s FROM '(\d+)')`

One rule, both languages

```
# pandas
calls['band'] = np.select([calls['talk_s'] == 0, calls['talk_s'] <= 30], ['No talk', 'Short'], default='Long')
```

```
-- PostgreSQL
CASE WHEN talk_s = 0  THEN 'No talk'
     WHEN talk_s <= 30 THEN 'Short'
     ELSE 'Long' END AS band
```

When a calculation can run in either place, run it where the data already is. Group and filter in SQL before exporting large tables; use pandas for steps that combine files or need Python.

Practice tasks

Use the mastery workbook's practice data (`D = load()`), so every task runs without real files. Write each answer without `apply` unless the task says so, then time it with `%timeit`.

Lambdas in the right places

- [ ] List the files in `P['stringee_dir']`, smallest first, using `sorted(..., key=lambda f: ...)`.
- [ ] Clean every column name of `D['ark']` to lowercase with underscores, using `rename(columns=lambda c: ...)`.
- [ ] In one chain on `D['calls']`: add `talk_s` and `connected` with `.assign(lambda ...)`, keep connected calls with `.loc[lambda ...]`, and count them per `CRM ID`.

- [ ] Per agent, list the distinct numbers they called as one text cell, using `agg` with `lambda s: ', '.join(...)`.

Replace the lambda

- [ ] Rewrite `D['calls']['talk_s'].apply(lambda x: 'Long' if x > 60 else 'Short')` with `np.where`, and time both.
- [ ] Rewrite a three-band `if/elif` function as `np.select`, then as `pd.cut`. Check all three give identical labels.
- [ ] Rewrite `df.apply(lambda r: r['Queue'] + r['Answer'], axis=1)` as a column sum on Stringee data.
- [ ] Get the second number from `D['strings']['caller_ids']` three ways: `apply`, `.str.split`, `.str.replace` with regex. Time each.

Power functions

- [ ] Ledger: per client, `txn_no`, `balance_change` and `days_since_prev`, using `cumcount`, `diff` and `shift`.
- [ ] Ledger: bills per day with a 7-day moving average (`pd.Grouper` + `rolling`).
- [ ] Calls: the 3 longest connected calls per agent (`sort_values` + `groupby().head(3)`).
- [ ] Customers: a `crosstab` of Manager by Oct Trading with totals, then as shares per manager.
- [ ] Merge `D['calls_all']` with `D['team']` using `indicator=True`, and list the unmatched dialer names.
- [ ] Melt the hourly pivot (workbook exercise 64) to long format and find each agent's busiest slot.

Put it together

- [ ] Rebuild the October traded-clients report (flag, Manager and agent pivot, totals) as **one method chain** with no intermediate variables, using `peek` to print row counts.